P1072R5
basic_string::resize_default_init

Published Proposal, 2019-10-07

This version:
http://wg21.link/P1072R5

Authors:
Chris Kennelly (Google)
Mark Zeren (VMware)

Audience:
LEWG, LWG, SG16

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

Allow access to default-initialized elements of basic_string.

Table of Contents

References

Informative References

§ 1. Motivation

Performance sensitive code is impacted by the cost of initializing and manipulating strings. When streaming data into a `basic_string`, a programmer is faced with an unhappy choice:

- **Pay for extra initialization** — `resize`, which zero initializes, followed by copy.
- **Pay for extra copies** — Populate a temporary buffer, copy it to the string.
- **Pay for extra "bookkeeping"** — `reserve` followed by small appends, each of which checks capacity and null terminates.

The effects of these unhappy choices can all be measured at scale, yet C++'s hallmark is to leave no room between the language (or in this case the library) and another language.

LEWGI polled on this at the [SAN] Meeting:

> We should promise more committee time to [P1072R1], knowing that our time is scarce and this will leave less time for other work?
> *Unanimous consent*

LEWG at the [SAN] Meeting:

> We want to solve this problem
>
> | SF | F | N | A | SA |
> |---|---|---|---|---|
> | 17 | 5 | 0 | 0 | 0 |
>
> Willing to solve this for string without solving it for vector
>
> | SF | F | N | A | SA |
> |---|---|---|---|---|
> | 6 | 9 | 2 | 1 | 0 |

§ 2. Proposal

This proposal addresses the problem by adding `basic_string::resize_default_init`:

```
template<typename Operation>
void resize_default_init(size_type n, Operation op);
```

8. *Effects:* Let `o = size()`. Alters the value of `*this` as follows:

    1. — If `n <= size()`, erases the last `size() - n` elements.
    2. — If `n > size()`, appends `n - size()` default-initialized elements.

    Invokes `erase(begin() + op(data(), n), end())`.

9. *Remarks:* If `op` throws the behavior is undefined.

    Let `m = op(data(), n)`.

    `m <= n` otherwise the behavior is undefined.

    If `m > o`, `op` shall replace ([**expr.ass**]) the values stored in the character array `[data()+o, data()+m)`. Until replaced, the values may be indeterminate [**basic.indet**] (6.6.4). [*Note:* `*(data()+o)` may not be `CharT()`. - *end note*]

    When `op` is called `*this` is in an unspecified state. `op` shall not bind or in any way access `*this`

    `op` shall not allow its first argument to be accessible after it returns.

In order to enable `resize_default_init`, this proposal makes it implementation-defined whether `basic_string` uses `allocator_traits::construct` and `allocator_traits::destroy` to construct and destroy the "char-like objects" that it controls. See § 6.2 Allocator Interaction and § 9 Wording below for more details.

Note that this is not currently proposed as `constexpr` member function template, however, if that is possible it is desirable.

§ 3. Proposal with Exceptions

The version of this proposal seen by LEWG in [Cologne] throws `std::range_error` when `op` returns a value greater than `n`. The authors now recommend UB as proposed above, but the exception-full proposal would be:

```
template<typename Operation>
void resize_default_init(size_type n, Operation op);
```

8. *Effects:* Let `o = size()`. Alters the value of `*this` as follows:

   1. — If `n <= size()`, erases the last `size() - n` elements.
   2. — If `n > size()`, appends `n - size()` default-initialized elements.

   Invokes `erase(op(data(), n))`.

9. *Throws:* `std::range_error` if `m > n`.

10. *Remarks:* Let `m = op(data(), n)`.

    If `m > o`, `op` shall replace ([**expr.ass**]) the values stored in the character array `[data()+o, data()+m)`. Until replaced, the values may be indeterminate [**basic.indet**] (6.6.4). [*Note:* `*(data()+o)` may not be `CharT()`. - *end note*]

    When `op` is called `*this` is in an unspecified state. `op` shall not bind or in any way access `*this`

    `op` shall not allow its first argument to be accessible after it returns.

    If `op` modifies the values in `[0, o)` and then eiher `op` throws or `m > n`, the modifications may be visible. Otherwise, the strong exception guarantee holds.

The final Remark is one motivation for preferring UB.

§ 4. Implementation

`libc++` includes a private implementation of a prior version of this proposal (based on an earlier revision) and uses it to avoid a dynamic allocation in `std::filesystem` [LIBC++].

§ 5. Examples

§ 5.1. Stamping a Pattern

Consider writing a pattern several times into a string:

```cpp
std::string GeneratePattern(const std::string& pattern, size_t count) {
   std::string ret;

   ret.reserve(pattern.size() * count);
   for (size_t i = 0; i < count; i++) {
     // SUB-OPTIMAL:
     // * Writes 'count' nulls
     // * Updates size and checks for potential resize 'count' times
     ret.append(pattern);
   }

   return ret;
}
```

Alternatively, we could adjust the output string's size to its final size, avoiding the bookkeeping in `append` at the cost of extra initialization:

```cpp
std::string GeneratePattern(const std::string& pattern, size_t count) {
   std::string ret;

   const auto step = pattern.size();
   // SUB-OPTIMAL: We memset step*count bytes only to overwrite them.
   ret.resize(step * count);
   for (size_t i = 0; i < count; i++) {
     // GOOD: No bookkeeping
     memcpy(ret.data() + i * step, pattern.data(), step);
   }

   return ret;
}
```

With this proposal:

```
std::string GeneratePattern(const std::string& pattern, size_t count) {
   std::string ret;

   const auto step = pattern.size();
   // GOOD: No initialization
   ret.resize_default_init(step * count, [&](char* buf, size_t n) {
       for (size_t i = 0; i < count; i++) {
         // GOOD: No bookkeeping
         memcpy(buf + i * step, pattern.data(), step);
       }
       return step * count;
   });

   return ret;
}
```

§ 5.2. Interacting with C

Consider wrapping a C API while working in terms of C++'s `basic_string` vocabulary. We *anticipate* over-allocating, as computation of the *length* of the data is done simultaneously with the computation of the *contents*.

```
extern "C" {
  int compress(void* out, size_t* out_size, const void* in, size_t in_size)
  size_t compressBound(size_t bound);
}

std::string CompressWrapper(std::string_view input) {
    std::string compressed;
    // Compute upper limit of compressed input.
    size_t size = compressBound(input.size());

    // SUB-OPTIMAL: Extra initialization
    compressed.resize(size);
    int ret = compress(&*compressed.begin(), &size, input.data(), input.siz
    if (ret != OK) {
      throw ...some error...
    }

    // Set actual size.
    compressed.erase(size);
    return compressed;
}
```

With this proposal:

```
extern "C" {
  int compress(void* out, size_t* out_size, const void* in, size_t in_size)
  size_t compressBound(size_t bound);
}

std::string CompressWrapper(std::string_view input) {
    std::string compressed;
    // Compute upper limit of compressed input.
    size_t size = compressBound(input.size());

    // GOOD: No initialization
    compressed.resize_default_init(size, [&](char* buf, size_t n) {
        int ret = compress(buf, &size, input.data(), input.size());
        if (ret != OK) {
          throw ...some error...
        }
        return n;
    }
    return compressed;
}
```

§ 6. Design Considerations

§ 6.1. Method vs. Alternatives

`resize_default_init` exposes users to UB if they read indeterminate ([**basic.indet**] (6.6.4)) values in the string buffer passed to `op`. Despite this foot-gun, `resize_default_init` is appealing because:

- Uninitialized buffers are not uncommon in C++. See for example `new T[20]` or the recently adopted `make_unique_default_init`.
- Dynamic analyzers like Memory Sanitizer [MSan] and Valgrind [Valgrind] can catch uninitialized reads.

During the [SAN] Meeting LEWG expressed a preference for implementing this functionality as a new method on `basic_string` (as proposed in [P1072R0]) rather than a standalone "storage buffer" type (one option in [P1072R1]):

> Method on string vs storage_buffer type:
>
> | Strong Method | Method | Neutral | Type | Strong Type |
> |---|---|---|---|---|
> | 9 | 2 | 3 | 2 | 2 |

During the [KOA] Meeting, LEWG expressed a preference for not weakening `basic_string`'s invariants after `resize_default_init` returns. Under this revision:

- A default-initialized buffer is *only* accessible to `op`.
- Under penalty of undefined behavior, `op` must assign (replace) the values in resulting buffer. The default initialized portion (`[data()+m, data()+n)`) above is erased and inaccessible to the program without further violation of class invariants.
- This proposal follows the "lambda" option discussed in [SAN].

Several other alternatives were discussed at [SAN], [KOA], and on the reflector. Please see § 7 Alternatives Considered below for more details.

§ 6.2. Allocator Interaction

Unlocking the desired optimizations requires *some* change to `basic_string`'s interaction with allocators. This proposal does what we think is the simplest possible change: remove the requirement that `basic_string` call `allocator_traits::construct` or `allocator_traits::destroy`.

This restriction should be acceptable because `basic_string` is defined to only hold "non-array trivial standard-layout" types. [**strings.general**] p1:

> This Clause describes components for manipulating sequences of any non-array trivial standard-layout (6.7) type. Such types are called *char-like types*, and objects of *char-like types* are *called char-like objects* or simply *characters*.

Removing calls to `construct` and `destroy` is compatible with `pmr::basic_string` as long as `uses_allocator_v<charT>` is `false`, which should be the case in practice.

Along the way, this proposal clarifies ambiguous language in [**string.require**] and [**container.requirements.general**] by:

- Stating explicitly that `basic_string` is allocator-aware.
- Introducing the term "allocator-aware" earlier in [**container.requirements.general**].

- Not attempting to mention other "components" in [**container.requirements.general**]. The other allocator-aware containers (just `basic_string` and `regex::match_results`?) can independently state that they are "allocator-aware" in their own clauses.

libstdc++ and msvc allow strings of non-trivial type. That might force those libraries to continue to support `construct` and `destroy` as "extensions". On the other hand, libc++ disallows non-trivial `charT`s, so any such extensions are non-portable. See godbolt.org.

§ 6.3. Bikeshedding

What do we want to call this method?

- `resize_default_init` (as proposed). This is consistent with `make_unique_default_init` [P1020R1], adopted in San Diego. Also, specifying default-initialization throws a bone to libraries that have historically allowed non-trivial `charT` types in `basic_string`.
- `resize_uninitialized` (as proposed in R0). "Uninitialized" is different from "default-initialized", which is what we want to specify. Also, `uninitialized` is already used elsewhere in the Library to mean "not constructed at all", as in `uninitialized_copy`; so this would be an overloading of the term.

§ 7. Alternatives Considered

§ 7.1. Tag Type

At the [SAN] Meeting, LEWG showed support for a tag argument type:

> Approval (vote for as many as you find acceptable)
>
> | | |
> |---|---|
> | 13 | **Go back to resize_uninitialized** |
> | 15 | **Do tag type (default_initialize) for c'tor / resize()** |
> | 12 | **Continue with storage_buffer (as per R2 of this paper)** |
> | 7 | **Crack open with a lambda** |
> | 7 | **RAII separate type** |

For example:

```
std::string GeneratePattern(const std::string& pattern, size_t count) {
   const auto step = pattern.size();

   // GOOD:  No initialization
   std::string ret(step * count, std::string::default_init);
   for (size_t i = 0; i < count; i++) {
     // GOOD: No bookkeeping
     memcpy(ret.data() + i * step, pattern.data(), step);
   }

   return ret;
}
```

Benefits:

- A constructor that takes a tag type simplifies some use cases, like the example above.
- Matches existing practice in Boost. See § 8.7 Boost.

Drawbacks:

- Feature creep / complexity — A tag type invites generalizing for all allocator-aware container types. We agree that this is desirable and even has implementation experience in Boost. However default initialization is not enough. There is also "implicit construction" (see [P1010R1], [P0593R2]) and "relocation" (see [P1144R0], [P1029R1]). Neither of these features are yet in the language. It is too early to generalize. Note that the second poll quoted in § 1 Motivation shows support for solving this problem for `[basic_]string` but not `vector`.
- In reflector discussion of `make_unique_default_init` [Zero], there was a preference for avoiding tag types. The standard library has `copy_backward`, not `copy` with a tag, and `count_if`, rather than `count` with a predicate.

Conclusion:

LEWG should explore tags for allocator-aware containers, but that work should not block near-term enablement of efficient [std::]string builders.

§ 7.2. Non-Lambda Approach

In [P1072R0] and [P1072R3] of this proposal, the authors considered a method `resize_default_init` / `resize_uninitialized` which left a default-initialized buffer accessible to users of the `basic_string` instance after the method returned. This method was rejected in [KOA], due to the weakened class invariants.

For illustration:

```
extern "C" {
  int compress(void* out, size_t* out_size, const void* in, size_t in_size)
  size_t compressBound(size_t bound);
}

std::string CompressWrapper(std::string_view input) {
    std::string compressed;
    // Compute upper limit of compressed input.
    size_t size = compressBound(input.size());

    // GOOD: No initialization
    compressed.resize_default_init(size);
    int ret = compress(&*compressed.begin(), &size, input.data(), input.siz
    if (ret != OK) {
      throw ...some error...
    }

    // Suppose size is the value of size before the call to compress and si
    // is the value of size after the call to compress.
    //
    // If size' < size, then:
    //   std::cout << compressed[size' + 1]
    // ...triggers a read of uninitialized data.

    // Set actual size.
    compressed.erase(size);
    return compressed;
}
```

§ 7.3. Standalone Type: `storage_buffer`

In [P1072R1], we considered `storage_buffer`, a standalone type providing a `prepare` method (similar to the `resize_uninitialized` method proposed here) and `commit` (to promise, under penalty of UB, that the buffer had been initialized).

At the [SAN] Meeting, this approach received support from LEWGI in light of the [post-Rapperswil] email review indicating support for a distinct type. This approach was rejected by the larger LEWG room in San Diego Meeting Diego.

The proposed type would be move-only.

```
std::string GeneratePattern(const std::string& pattern, size_t count) {
   std::storage_buffer<char> tmp;

   const auto step = pattern.size();
   // GOOD:  No initialization
   tmp.prepare(step * count + 1);
   for (size_t i = 0; i < count; i++) {
     // GOOD: No bookkeeping
     memcpy(tmp.data() + i * step, pattern.data(), step);
   }

   tmp.commit(step * count);
   return std::string(std::move(tmp));
}
```

For purposes of the container API, `size()` corresponds to the *committed* portion of the buffer. This leads to more consistency when working with (and explicitly copying to) other containers via iterators, for example:

```
std::storage_buffer<char> buf;
buf.prepare(100);
*fill in data*
buf.commit(50);

std::string a(buf.begin(), buf.end());
std::string b(std::move(buf));

assert(a == b);
```

Benefits:

- By being move-only, we would not have the risk of copying types with trap representations (thereby triggering UB).

- Uninitialized data is only accessible from the `storage_buffer` type. For an API working with `basic_string`, no invariants are weakened. Crossing an API boundary with a `storage_buffer` is much more obvious than a "`basic_string` with possibly uninitialized data." Uninitialized bytes (the promise made by `commit`) never escape into the valid range of `basic_string`.

Drawbacks:

- `storage_buffer` requires introducing an extra type to the standard library, even though its novel functionality (from `string` and `vector`) is limited to the initialization abilities.
- `basic_string` is often implemented with a short-string optimization (SSO) and an extra type would need to implement that (likely by additional checks when moving to/from the `storage_buffer`) that are often unneeded.

§ 7.4. Externally-Allocated Buffer Injection

In [P1072R1], we considered that `basic_string` could "adopt" an externally `allocator::allocate`'d buffer. At the [SAN] Meeting, we concluded that this was:

- **Not critical**
- **Not constrained in the future**
- **Overly constraining to implementers**. Allowing users to provide their own buffers runs into the "offset problem". Consider an implementation that stores its `size` and `capacity` inline with its data, so `sizeof(container) == sizeof(void*)`.

  ```
  class container {
    struct Rep {
      size_t size;
      size_t capacity;
    };

    Rep* rep_;
  };
  ```

  If using a `Rep`-style implementation, the mismatch in offsets requires an O(N) move to shift the contents into place and trigger a possible reallocation.
- **Introducing new pitfalls**. It would be easy to mix `new[]` and `allocator::allocate` inadvertently.

§ 8. Related Work

§ 8.1. Google

Google has a local extension to `basic_string` called `resize_uninitialized` which is wrapped as `STLStringResizeUninitialized`.

- [Abseil] uses this to avoid bookkeeping overheads in `StrAppend` and `StrCat`.
- [Snappy]
    - In decompression, the final size of the output buffer is known before the contents are ready.
    - During compression, an upperbound on the final compressed size is known, allowing data to be efficiently added to the output buffer (eliding `append`'s checks) and the string to be shrunk to its final, correct size.
- [Protobuf] avoids extraneous copies or initialization when the size is known before data is available (especially during parsing or serialization).

§ 8.2. MongoDB

MongoDB has a string builder that could have been implemented in terms of `basic_string` as a return value. However, as explained by Mathias Stearn, zero initialization was measured and was too costly. Instead a custom string builder type is used:

E.g.: https://github.com/mongodb/mongo/blob/master/src/mongo/db/fts/unicode/string.h

```
/**
 * Strips diacritics and case-folds the utf8 input string, as needed to sup
 * options.
 *
 * The options field specifies what operations to *skip*, so kCaseSensitive
 * means to skip case folding and kDiacriticSensitive means to skip diacrit
 * striping. If both flags are specified, the input utf8 StringData is retu
 * directly without any processing or copying.
 *
 * If processing is performed, the returned StringData will be placed in
 * buffer.  buffer’s contents (if any) will be replaced. Since we may retur
 * the input unmodified the returned StringData’s lifetime is the shorter o
 * the input utf8 and the next modification to buffer. The input utf8 must
 * point into buffer.
 */
static StringData caseFoldAndStripDiacritics(StackBufBuilder* buffer,
                                             StringData utf8,
                                             SubstrMatchOptions options,
                                             CaseFoldMode mode);
```

(Comments re-wrapped.)

§ 8.3. VMware

VMware has an internal string builder implementation that avoids `std::string` due, in part, to `reserve`'s zero-writing behavior. This is similar in spirit to the MongoDB example above.

§ 8.4. Discussion on std-proposals

This topic was discussed in 2013 on std-proposals in a thread titled "Add basic_string::resize_uninitialized (or a similar mechanism)": https://groups.google.com/a/isocpp.org/forum/#!topic/std-proposals/XIO4KbBTxl0

§ 8.5. DynamicBuffer

The [N4734] (the Networking TS) has *dynamic buffer* types.

8.6. P1020R1

See also [P1020R1] "Smart pointer creation functions for default initialization". Adopted in San Diego.

8.7. Boost

Boost provides a related optimization for vector-like containers, introduced in [SVN r85964] by Ion Gaztañaga.

E.g.: boost/container/vector.hpp:

```
//! <b>Effects</b>: Constructs a vector that will use a copy of allocator a
//!   and inserts n default initialized values.
//!
//! <b>Throws</b>: If allocator_type’s allocation
//!   throws or T’s default initialization throws.
//!
//! <b>Complexity</b>: Linear to n.
//!
//! <b>Note</b>: Non-standard extension
vector(size_type n, default_init_t);
vector(size_type n, default_init_t, const allocator_type &a)
...
void resize(size_type new_size, default_init_t);
...
```

These optimizations are also supported in Boost Container’s `small_vector`, `static_vector`, `deque`, `stable_vector`, and `string`.

8.8. Thrust

The Thrust library has "a RAII-type `thrust::detail::temporary_array` which has a vector-like interface and a constructor with a tag parameter that indicates its elements should not be initialized." - [Bryce Adelstein Lelbach].

E.g. thrust/thrust/detail/temporary_array.inl:

```
template<typename T, typename TemporaryArray, typename Size>
__host__ __device__
typename thrust::detail::disable_if<
  avoid_initialization<T>::value
>::type
  construct_values(TemporaryArray &a,
                   Size n)
{
  a.default_construct_n(a.begin(), n);
} // end construct_values()
```

§ 9. Wording

Wording is relative to [N4830].

Motivation for some of these edits can be found in § 6.2 Allocator Interaction.

§ 9.1. [**basic.string**]

In [**basic.string**] [21.3.2], in the synopsis, add `resize_default_init`:

```
...
    // 21.3.2.4, capacity
    constexpr size_type size() const noexcept;
    constexpr size_type length() const noexcept;
    constexpr size_type max_size() const noexcept;
    constexpr void resize(size_type n, charT c);
    constexpr void resize(size_type n);
    template<typename Operation>
    void resize_default_init(size_type n, Operation op);
    constexpr size_type capacity() const noexcept;
    constexpr void reserve(size_type res_arg);
    constexpr void shrink_to_fit();
    constexpr void clear() noexcept;
    [[nodiscard]] constexpr bool empty() const noexcept;
```

In [**string.require**] [21.3.2.1] clarify that `basic_string` is an allocator-aware container, and then add an exception for `construct` and `destroy`.

3. In every specialization `basic_string<charT, traits, Allocator>`, the type `allocator_traits<Allocator>::value_type` shall name the same type as `charT` ~~.~~ ~~Every object of type `basic_string<charT, traits, Allocator>` uses an object of type `Allocator` to allocate and free storage for the contained `charT` objects as needed. The Allocator object used is obtained as described in [container.requirements.general]. In every specialization `basic_string<charT, traits, Allocator>`~~ , and the type `traits` shall satisfy the character traits requirements ([char.traits]). [*Note:* The program is ill-formed if `traits::char_type` is not the same type as `charT`. — *end note*]

4. `basic_string` is an allocator-aware container as described in [container.requirements.general], except that it is implementation-defined whether `allocator_traits::construct` and `allocator_traits::destroy` are used to construct and destroy the contained `charT` objects.

5. References, pointers, and iterators referring to the elements of a `basic_string` sequence may be invalidated by the following uses of that `basic_string` object:
   ...

In [**string.capacity**] [21.3.2.4]:

```
constexpr void resize(size_type n, charT c);
```

6. *Effects:* Alters the value of `*this` as follows:
    1. — If `n <= size()`, erases the last `size() - n` elements.
    2. — If `n > size()`, appends `n - size()` copies of `c`.

```
constexpr void resize(size_type n);
```

7. *Effects:* Equivalent to `resize(n, charT())`.

```
template<typename Operation>
void resize_default_init(size_type n, Operation op);
```

8. *Effects:* Let `o = size()`. Alters the value of `*this` as follows:
    1. — If `n <= size()`, erases the last `size() - n` elements.
    2. — If `n > size()`, appends `n - size()` default-initialized elements.

    Invokes `erase(begin() + op(data(), n), end())`.
9. *Remarks:* If `op` throws the behavior is undefined.

    Let `m = op(data(), n)`.

    `m <= n` otherwise the behavior is undefined.

    If `m > o`, `op` shall replace ([**expr.ass**]) the values stored in the character array `[data()+o, data()+m)`. Until replaced, the values may be indeterminate [**basic.indet**] (6.6.4). [*Note:* `*(data() + o)` may not be `CharT()`. - *end note*]

    When `op` is called `*this` is in an unspecified state. `op` shall not bind or in any way access `*this`

    `op` shall not allow its first argument to be accessible after it returns.

```
constexpr size_type capacity() const noexcept;
```

...

§ 9.2. [**container.requirements.general**]

In [**container.requirements.general**] [22.2.1] clarify the ambiguous "components affected by this subclause" terminology in p3. Just say "allocator-aware containers".

> 3. All of the containers defined in this Clause except `array` are allocator-aware containers. ~~For the components affected by this subclause that declare an allocator_type~~ Objects ~~objects~~ stored in ~~these components~~ allocator-aware containers, unless otherwise specified, shall be constructed using the function `allocator_traits<allocator_type>::rebind_traits<U>::construct` and destroyed using the function `allocator_traits<allocator_type>::rebind_traits<U>::destroy` (20.10.9.2), where `U` is either `allocator_type::value_type` or an internal type used by the container.
>
> ...

We can then simplify p15:

> 15. ~~All of the containers defined in this Clause and in 21.3.2 except `array` meet the additional requirements of an allocator-aware container, as~~ Allocator-aware containers meet the additional requirements described in Table 75.

§ 10. Questions for LEWG

1. Should `op` returning a size larger than requested initially throw or result in UB?
2. Should `resize_default_init` be `constexpr`?

§ 11. History

§ 11.1. R4 → R5

A draft of this revision was presented to LEWG at the [Cologne] meeting.

> Forward to LWG for C++23
>
> | SF | F | N | A | SA |
> |---|---|---|---|---|
> | 10 | 5 | 1 | 0 | 0 |

- Rebased on [N4830].
- Proposed UB instead of `throwing std::range_error`.

§ 11.2. R3 → R4

- Applied feedback from the [KOA] meeting.
- Moved to using a callback routine, rather than leaving invariants weakened from the time `resize_default_init` returned until values in the buffer were assigned.

§ 11.3. R2 → R3

- Applied Jonathan Wakely’s editorial comments on § 9 Wording.
- Rebased on [N4791].
- Editorial changes to § 1 Motivation and § 7.1 Tag Type to support our design choice.
- Added the reference to [LIBC++] in § 4 Implementation.

§ 11.4. R1 → R2

Applied feedback from [SAN] Meeting reviews.

- Reverted design to "option A" proposed in [P1072R0].
- Switched from `resize_uninitialized` to `resize_default_init`.
- Added discussion of alternatives considered.
- Specified allocator interaction.
- Added wording.

§ 11.5. R0 → R1

Applied feedback from LEWG [post-Rapperswil] Email Review:

- Shifted to a new vocabulary types: `storage_buffer` / `storage_node`
    - Added presentation of `storage_buffer` as a new container type
    - Added presentation of `storage_node` as a node-like type
- Added discussion of design and implementation considerations.
- Most of [P1010R1] Was merged into this paper.

§ 12. Acknowledgments

Thanks go to **Arthur O’Dwyer** for help with wording and proof reading, to **Jonathan Wakely** for hunting down the language that makes `basic_string` allocator-aware, and to **Glen Fernandes**, **Corentin Jabot**, **Billy O’Neal**, and **Mathias Stearn** for design discussion. Special thanks to **Eric Fiselier** for providing the implmentation.

§ References

§ Informative References

[Abseil]
Abseil. 2018-09-22. URL: https://github.com/abseil/abseil-cpp

[Cologne]
Cologne Meeting Minutes. 2019-07-19. URL: http://wiki.edg.com/bin/view/Wg21cologne2019/P1072

[KOA]
Kona Meeting Minutes. 2019-02-19. URL: http://wiki.edg.com/bin/view/Wg21kona2019/P1072

[LIBC++]
Add basic_string::__resize_default_init (from P1072). 2018-11-26. URL: http://llvm.org/viewvc/llvm-project?view=revision&revision=347589

[MSan]
Memory Sanitizer. URL: https://clang.llvm.org/docs/MemorySanitizer.html

[N4734]
Jonathan Wakely. Working Draft, C ++ Extensions for Networking. 4 April 2018. URL: https://wg21.link/n4734

[N4791]
Richard Smith. Working Draft, Standard for Programming Language C++. 26 November 2018. URL: https://wg21.link/n4791

[N4830]
Richard Smith. Committee Draft, Standard for Programming Language C++. 15 August 2019. URL: https://wg21.link/n4830

[P0593R2]
Richard Smith. Implicit creation of objects for low-level object manipulation. 11 February 2018. URL: https://wg21.link/p0593r2

[P1010R1]
Mark Zeren, Chris Kennelly. Container support for implicit lifetime types. 8 October 2018. URL: https://wg21.link/p1010r1

[P1020R1]
Glen Joseph Fernandes, Peter Dimov. Smart pointer creation with default initialization. 6 November 2018. URL: https://wg21.link/p1020r1

[P1029R1]
Niall Douglas. [[move_relocates]]. 7 August 2018. URL: https://wg21.link/p1029r1

[P1072R0]
Chris Kennelly, Mark Zeren. Default Initialization for basic_string. 4 May 2018. URL: https://wg21.link/p1072r0

[P1072R1]
Chris Kennelly, Mark Zeren. Optimized Initialization for basic_string and vector. 7 October 2018. URL: https://wg21.link/p1072r1

[P1072R3]
Chris Kennelly, Mark Zeren. basic_string::resize_default_init. 21 January 2019. URL: https://wg21.link/p1072r3

[P1144R0]
Arthur O'Dwyer, Mingxin Wang. Object relocation in terms of move plus destroy. 4 October 2018. URL: https://wg21.link/p1144r0

[post-Rapperswil]
LEWG Weekly - P1072. 2018-08-26. URL: http://lists.isocpp.org/lib-ext/2018/08/8299.php

[Protobuf]
Protocol Buffers. 2018-09-22. URL: https://github.com/protocolbuffers/protobuf

[SAN]
San Diego Meeting Minutes. 2018-11-07. URL: http://wiki.edg.com/bin/view/Wg21sandiego2018/P1072

[Snappy]
Snappy. 2018-09-21. URL: https://github.com/google/snappy

[Valgrind]
Valgrind. URL: http://www.valgrind.org

[Zero]
Listening to our customers: Zero-initialization issue. 2018-04-13. URL: http://lists.isocpp.org/lib-ext/2018/04/6712.php